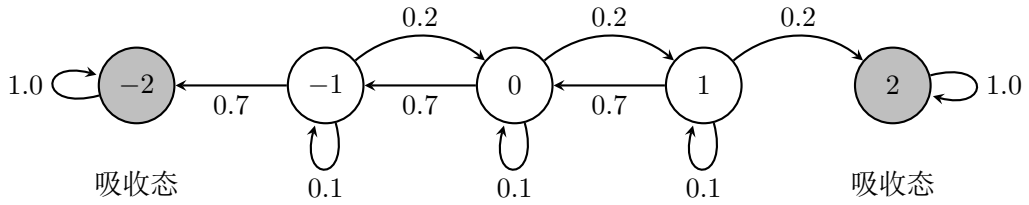
# DS4001-25SP-HW2：搜索

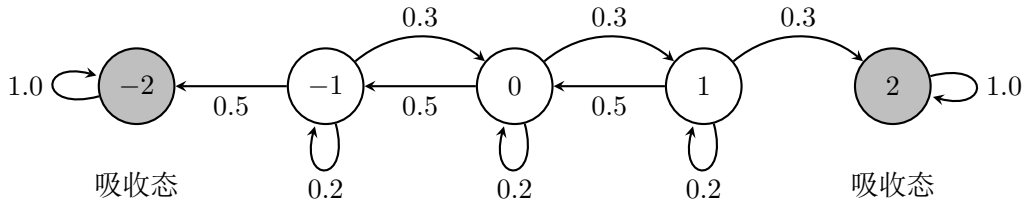
黄思睿 PB23151782　PB

2025 年 4 月 21 日

## 1　问题 1：马尔可夫决策过程 [9%=6%+3%]




$a_1$ 策略转移图




$a_2$ 策略转移图

(a) 如上图，分别绘制出 $a_1$ 策略和 $a_2$ 策略的状态转移图

代入价值迭代公式来求状态价值函数。公式如下：

$$V^{(i+1)}(s) = \max_a \sum_{s'} P(s'|s,a) \cdot [R(s,a,s') + \gamma \cdot V^{(i)}(s')]$$

其中，$V^{(0)}(s) \equiv 0$ ，$\gamma = 1$。

**计算 $V^{(1)}(s)$：** 对于吸收状态：

$$V^{(1)}(-2) = 0 \tag{1}$$

$$V^{(1)}(2) = 0 \tag{2}$$

对于 $s = -1$:

$$V^{(1)}(-1) = \max \begin{cases} a_1 : 0.7 \cdot (10+0) + 0.1 \cdot (-1+0) + 0.2 \cdot (-1+0) = 7 - 0.1 - 0.2 = 6.7 \\ a_2 : 0.5 \cdot (10+0) + 0.2 \cdot (-1+0) + 0.3 \cdot (-1+0) = 5 - 0.2 - 0.3 = 4.5 \end{cases} \tag{3}$$

$$= 6.7 \tag{4}$$

对于 $s = 0$:

$$V^{(1)}(0) = \max \begin{cases} a_1 : 0.7 \cdot (-1+0) + 0.1 \cdot (-1+0) + 0.2 \cdot (-1+0) = -0.7 - 0.1 - 0.2 = -1.0 \\ a_2 : 0.5 \cdot (-1+0) + 0.2 \cdot (-1+0) + 0.3 \cdot (-1+0) = -0.5 - 0.2 - 0.3 = -1.0 \end{cases} \tag{5}$$

$$= -1.0 \tag{6}$$

对于 $s = 1$:

$$V^{(1)}(1) = \max \begin{cases} a_1 : 0.7 \cdot (-1+0) + 0.1 \cdot (-1+0) + 0.2 \cdot (20+0) = -0.7 - 0.1 + 4 = 3.2 \\ a_2 : 0.5 \cdot (-1+0) + 0.2 \cdot (-1+0) + 0.3 \cdot (20+0) = -0.5 - 0.2 + 6 = 5.3 \end{cases} \tag{7}$$

$$= 5.3 \tag{8}$$

因此:

$$V^{(1)} = \{V^{(1)}(-2) = 0, V^{(1)}(-1) = 6.7, V^{(1)}(0) = -1.0, V^{(1)}(1) = 5.3, V^{(1)}(2) = 0\}$$

**计算 $V^{(2)}(s)$:** 对于吸收状态:

$$V^{(2)}(-2) = 0 \tag{9}$$

$$V^{(2)}(2) = 0 \tag{10}$$

对于 $s = -1$:

$$V^{(2)}(-1) = \max \begin{cases} a_1 : 0.7 \cdot (10+0) + 0.1 \cdot (-1+6.7) + 0.2 \cdot (-1+(-1)) \\ \quad = 7 + 0.57 - 0.4 = 7.17 \\ a_2 : 0.5 \cdot (10+0) + 0.2 \cdot (-1+6.7) + 0.3 \cdot (-1+(-1)) \\ \quad = 5 + 1.14 - 0.6 = 5.54 \end{cases} \tag{11}$$

$$= 7.17 \tag{12}$$

对于 $s = 0$:

$$V^{(2)}(0) = \max \begin{cases} a_1 : 0.7 \cdot (-1+6.7) + 0.1 \cdot (-1+(-1)) + 0.2 \cdot (-1+5.3) \\ \quad = 3.99 - 0.2 + 0.86 = 4.65 \\ a_2 : 0.5 \cdot (-1+6.7) + 0.2 \cdot (-1+(-1)) + 0.3 \cdot (-1+5.3) \\ \quad = 2.85 - 0.4 + 1.29 = 3.74 \end{cases} \tag{13}$$

$$= 4.65 \tag{14}$$

对于 $s = 1$:

$$V^{(2)}(1) = \max \begin{cases} a_1 : 0.7 \cdot (-1 + (-1)) + 0.1 \cdot (-1 + 5.3) + 0.2 \cdot (20 + 0) \\ \quad = -1.4 + 0.43 + 4 = 3.03 \\ a_2 : 0.5 \cdot (-1 + (-1)) + 0.2 \cdot (-1 + 5.3) + 0.3 \cdot (20 + 0) \\ \quad = -1 + 0.86 + 6 = 5.86 \end{cases} \tag{15}$$

$$= 5.86 \tag{16}$$

因此:

$$V^{(2)} = \{V^{(2)}(-2) = 0, V^{(2)}(-1) = 7.17, V^{(2)}(0) = 4.65, V^{(2)}(1) = 5.86, V^{(2)}(2) = 0\}$$

(b) 根据计算得到的 $V^{(2)}(s)$ 值，最优策略 $\mu(s)$ 是选择使价值最大化的动作。

对于 $s = -1$:

$$Q^{(2)}(-1, a_1) = 7.17 \tag{17}$$

$$Q^{(2)}(-1, a_2) = 5.54 \tag{18}$$

$$\therefore \mu(-1) = a_1 \tag{19}$$

对于 $s = 0$:

$$Q^{(2)}(0, a_1) = 4.65 \tag{20}$$

$$Q^{(2)}(0, a_2) = 3.74 \tag{21}$$

$$\therefore \mu(0) = a_1 \tag{22}$$

对于 $s = 1$:

$$Q^{(2)}(1, a_1) = 3.03 \tag{23}$$

$$Q^{(2)}(1, a_2) = 5.86 \tag{24}$$

$$\therefore \mu(1) = a_2 \tag{25}$$

因此，最优策略为:

$$\mu(-1) = a_1 \tag{26}$$

$$\mu(0) = a_1 \tag{27}$$

$$\mu(1) = a_2 \tag{28}$$

## 2 问题 2：Q-Learning[12%=3%+6%+3%]

(a) 我们需要从定义出发，推导 $Q(s, a)$ 与状态价值函数 $V(s')$ 的关系：

$$Q(s, a) = \mathbb{E}[G_t | s_t = s, a_t = a] \tag{29}$$

$$= \mathbb{E}[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots | s_t = s, a_t = a] \tag{30}$$

$$= \mathbb{E}[r_t + \gamma(r_{t+1} + \gamma r_{t+2} + \ldots) | s_t = s, a_t = a] \tag{31}$$

$$= \mathbb{E}[r_t + \gamma G_{t+1} | s_t = s, a_t = a] \tag{32}$$

进一步展开：

$$Q(s,a) = \mathbb{E}[r_t|s_t = s, a_t = a] + \gamma\mathbb{E}[G_{t+1}|s_t = s, a_t = a] \tag{33}$$

前一项是当前奖励的期望，后一项可按条件期望公式展开：

$$Q(s,a) = \sum_{s'} P(s'|s,a)R(s,a,s') + \gamma \sum_{s'} P(s'|s,a)\mathbb{E}[G_{t+1}|s_{t+1} = s'] \tag{34}$$

$$= \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V(s')] \tag{35}$$

因此，$Q(s,a)$ 可以用状态价值函数 $V(s')$ 表示为：

$$Q(s,a) = \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V(s')]$$

(b) 对于轨迹 $\tau = \{(0,a_1,2),(1,a_1,3),(0,a_2,-1),(1,a_2,0)\}$，

假设学习率 $\alpha = 1$，衰减因子 $\gamma = 1$，

蒙特卡洛方法的更新公式为：$Q(s,a) = Q(s,a) + \alpha[G_t - Q(s,a)]$

当 $\alpha = 1$ 时，这个公式简化为：$Q(s,a) = G_t$

初始条件：所有 $q(s,a) = 0$

**对于 $t = 4$, $(1,a_2,0)$:**

$$G_4 = r_4 = 0 \tag{36}$$

$$q(1,a_2) = G_4 = 0 \tag{37}$$

**对于 $t = 3$, $(0,a_2,-1)$:**

$$G_3 = r_3 + \gamma G_4 = -1 + 1 \times 0 = -1 \tag{38}$$

$$q(0,a_2) = G_3 = -1 \tag{39}$$

**对于 $t = 2$, $(1,a_1,3)$:**

$$G_2 = r_2 + \gamma G_3 = 3 + 1 \times (-1) = 2 \tag{40}$$

$$q(1,a_1) = G_2 = 2 \tag{41}$$

**对于 $t = 1$, $(0,a_1,2)$:**

$$G_1 = r_1 + \gamma G_2 = 2 + 1 \times 2 = 4 \tag{42}$$

$$q(0,a_1) = G_1 = 4 \tag{43}$$

**最终 Q 值:**

$$q(0, a_1) = 4 \tag{44}$$
$$q(1, a_1) = 2 \tag{45}$$
$$q(0, a_2) = -1 \tag{46}$$
$$q(1, a_2) = 0 \tag{47}$$

(c) 根据知乎上笔记中的相关证明，Q-Learning 算法能够收敛的原因简略如下：

(a) **贝尔曼最优算子的收缩性质**：
Q-Learning 基于贝尔曼最优方程 $Q^*(s,a) = R(s,a) + \gamma \sum_{s'} P(s'|s,a) \max_{a'} Q^*(s',a')$，其中贝尔曼算子 $T$ 是一个 $\gamma$-收缩映射，满足 $\|TQ_1 - TQ_2\| \le \gamma \|Q_1 - Q_2\|$。

(b) **误差收敛证明**：定义 $\Delta_n = \max_{s,a} |Q_n(s,a) - Q^*(s,a)|$ 为第 $n$ 次更新后的最大误差，可证明每次 Q 值更新后：

$$|Q_{n+1}(s,a) - Q^*(s,a)| \le (\alpha\gamma + (1-\alpha))\Delta_n \tag{48}$$

当 $\alpha \in (0,1]$ 且 $\gamma \in [0,1)$ 时，系数 $(\alpha\gamma + (1-\alpha)) < 1$，因此误差会随着更新次数增加而收缩。

(c) **遍历**：当每个状态-动作对 $(s,a)$ 都被访问无穷多次时，经过 $k$ 个" 遍历区间" 后，最大误差 $\Delta_n$ 不超过 $(\alpha\gamma + (1-\alpha))^k \Delta_0$，当 $k \to \infty$ 时，$\Delta_n \to 0$。

综上所述，Q-Learning 在有限状态和动作空间、有界奖励以及适当探索条件下，能够收敛到最优动作价值函数 $Q^*$，从而推导出最优策略 $\pi^*(s) = \arg\max_a Q^*(s,a)$。

# 3 问题 3：Gobang Programming[53%=33%+10%+10%]

(a) **[代码]**

```python
def get_next_state(self, action: Tuple[int, int, int], noise: Tuple[int, int, int]):
    # BEGIN_YOUR_CODE (our solution is 3 line of code, but don't worry if you deviate from this)
    """返回黑棋落子后的棋盘状态"""
    next_state = copy.deepcopy(self.board)
    color, x, y = action
    next_state[x][y] = color
    # END_YOUR_CODE

    if noise is not None:
        white, x_white, y_white = noise
        next_state[x_white][y_white] = white
    return next_state
```

```python
def sample_noise(self):
    if self.action_space:
        # BEGIN_YOUR_CODE (our solution is 2 line of code, but don't worry if you deviate from this)
        pos = random.choice(self.action_space)
            x, y = pos
            self.action_space.remove(pos)
        # END_YOUR_CODE
        return 2, x, y
    else:
        return None
```

```python
def get_connection_and_reward(self, action: Tuple[int, int, int], noise: Tuple[int, int, int]):
    # BEGIN_YOUR_CODE (our solution is 4 line of code, but don't worry if you deviate from this)
    """计算奖励值"""
    black_1, white_1 = self.count_max_connections(self.board)
    next_state = self.get_next_state(action, noise)
    black_2, white_2 = self.count_max_connections(next_state)
    reward = (black_2 - white_2) - (black_1 - white_1)
    # END_YOUR_CODE

    return black_1, white_1, black_2, white_2, reward
```

```python
def sample_action_and_noise(self, eps: float):
    # BEGIN_YOUR_CODE (our solution is 8 line of code, but don't worry if you deviate from this)
    """使用epsilon-greedy策略选择动作"""
    state = self.array_to_hashable(self.board)

    if not self.action_space:  # 如果没有可用动作
        return None, None

    # 决定是随机探索还是贪心选择
    if random.random() < eps or state not in self.Q:
        # 随机探索
        pos = random.choice(self.action_space)
        x, y = pos
        action = (1, x, y)
    else:
        # 贪心选择 - 找Q值最大的动作
        best_action = None
        best_q = -float('inf')

        for pos in self.action_space:
            x, y = pos
            act = (1, x, y)
            if act in self.Q[state] and self.Q[state][act] > best_q:
                best_q = self.Q[state][act]
                best_action = act

        if best_action:
            action = best_action
        else:
            # 如果没有找到最佳动作，随机选择
            pos = random.choice(self.action_space)
            x, y = pos
            action = (1, x, y)

    # 从动作空间移除选择的位置
    _, x, y = action
    self.action_space.remove((x, y))
    # END_YOUR_CODE
    return action, self.sample_noise()
```

```python
def q_learning_update(self, s0_: np.array, action: Tuple[int, int, int], s1_: np.array, reward: float,
                        alpha_0: float = 1):
    s0, s1 = self.array_to_hashable(s0_), self.array_to_hashable(s1_)
    self.s_a_visited[(s0, action)] = 1 if (s0, action) not in self.s_a_visited else \
        self.s_a_visited[(s0, action)] + 1
    alpha = alpha_0 / self.s_a_visited[(s0, action)]

```

```python
    # BEGIN_YOUR_CODE (our solution is 18 line of code, but don't worry if you deviate from this)
    # 初始化Q表
    if s0 not in self.Q:
        self.Q[s0] = {}
    if action not in self.Q[s0]:
        self.Q[s0][action] = 0.0

    # 找到下一个状态的最大Q值
    max_q_next = 0.0

    if s1 in self.Q and self.Q[s1]:
        max_q_next = max(self.Q[s1].values())

    # 更新公式
    self.Q[s0][action] = self.Q[s0][action] + alpha * (reward + self.gamma * max_q_next - self.Q[s0][action])
    # END_YOUR_CODE
```

(b) **[截图]**

图 1: n=3 训练截图

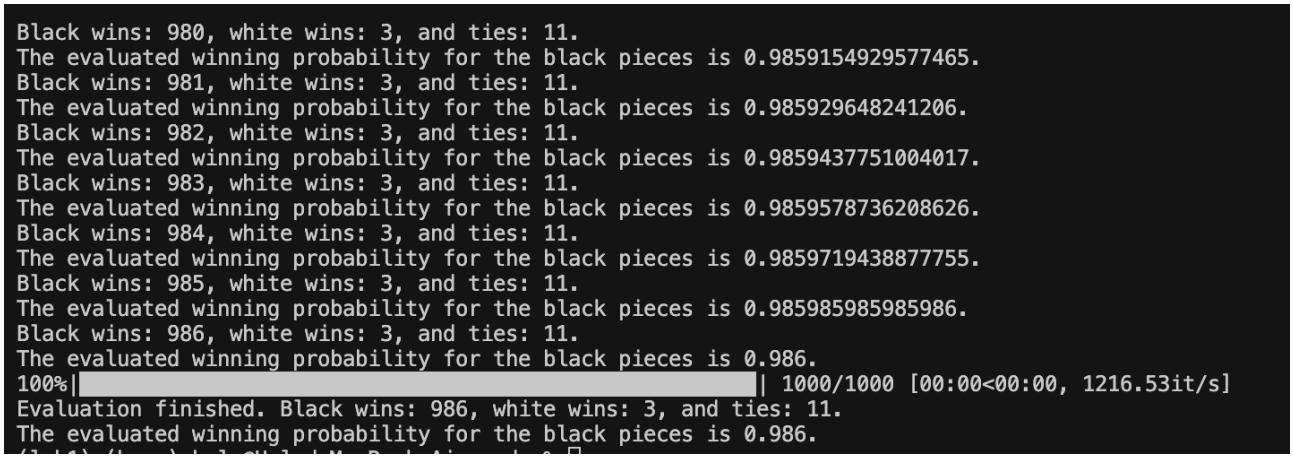



图 2: n=3 评估截图

(c) **[截图]**

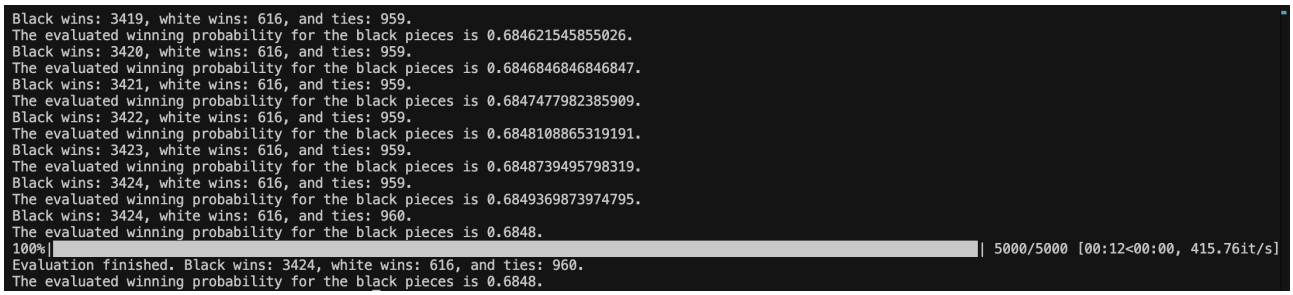



图 3: n=4 评估截图（结果不稳定，在胜率在 0.60 上下）

将 n=4 作为参数进行实验，在 $4 \times 4$ 的棋盘上尝试执行 Q-Learning 算法的结果分析：

这个结果与预期相符。在 $4 \times 4$ 棋盘上，智能体的胜率相对于 $3 \times 3$ 棋盘下降较多，且平局明显增多，原因如下：

(a) **状态空间爆炸**：$4 \times 4$ 棋盘的状态空间 ($3^{16}$) 远大于 $3 \times 3$ 棋盘 ($3^9$)，算法需要更多的训练才能探索到足够多的状态。

(b) **Q 表覆盖不足**：由于状态空间增大，在相同训练轮次下，智能体只能访问到更小比例的状态，导致 Q 表覆盖率下降，策略质量降低。

(c) **增加的复杂性**：$4 \times 4$ 棋盘上的 “三子棋” 游戏比 $3 \times 3$ 棋盘更复杂，有更多的获胜路径和防守策略，需要更深层次的战术考量。

(d) **随机对手的相对优势**：在更大的棋盘上，随机策略的不可预测性可能会对学习中的智能体造成更多干扰。

若要在 $4 \times 4$ 棋盘上获得更好的性能，建议增加训练轮次、降低学习率衰减速度并采用更复杂的探索策略，以更好地平衡探索与利用。

# 4 问题 4：Deeper Understanding[16%]

## 4.1 Bellman 算子与压缩映射

(a) 我们需要证明 Bellman 算子 $T$ 是压缩映射，即对于 $v_1, v_2 \in \mathbb{R}^{|S|}$，恒有 $\|Tv_1 - Tv_2\|_\infty \leq \gamma \|v_1 - v_2\|_\infty$。

根据 Bellman 算子定义：$(Tv)(s) = \max_{a \in A}\{r_{sa} + \gamma \cdot \sum_{s' \in S} p_{sas'} \cdot v(s')\}$

对于任意状态 $s$，假设 $a_1$ 是使 $(Tv_1)(s)$ 取最大值的动作，$a_2$ 是使 $(Tv_2)(s)$ 取最大值的动作：

$$(Tv_1)(s) = r_{sa_1} + \gamma \cdot \sum_{s' \in S} p_{sa_1s'} \cdot v_1(s') \tag{49}$$

$$(Tv_2)(s) = r_{sa_2} + \gamma \cdot \sum_{s' \in S} p_{sa_2s'} \cdot v_2(s') \tag{50}$$

根据最大化操作的性质，我们有：

$$(Tv_1)(s) \geq r_{sa_2} + \gamma \cdot \sum_{s' \in S} p_{sa_2s'} \cdot v_1(s') \tag{51}$$

$$(Tv_2)(s) \geq r_{sa_1} + \gamma \cdot \sum_{s' \in S} p_{sa_1s'} \cdot v_2(s') \tag{52}$$

因此：

$$(Tv_1)(s) - (Tv_2)(s) \leq \gamma \cdot \sum_{s' \in S} p_{sa_1s'} \cdot (v_1(s') - v_2(s')) \tag{53}$$

$$(Tv_2)(s) - (Tv_1)(s) \leq \gamma \cdot \sum_{s' \in S} p_{sa_2s'} \cdot (v_2(s') - v_1(s')) \tag{54}$$

对任意 $s'$，有 $|v_1(s') - v_2(s')| \leq \|v_1 - v_2\|_\infty$，因此：

$$|(Tv_1)(s) - (Tv_2)(s)| \leq \gamma \cdot \sum_{s' \in S} p_{sas'} \cdot |v_1(s') - v_2(s')| \tag{55}$$

$$\leq \gamma \cdot \|v_1 - v_2\|_\infty \cdot \sum_{s' \in S} p_{sas'} \tag{56}$$

$$= \gamma \cdot \|v_1 - v_2\|_\infty \tag{57}$$

这对任意 $s$ 成立，所以 $\|Tv_1 - Tv_2\|_\infty \leq \gamma \|v_1 - v_2\|_\infty$，证明 $T$ 是压缩映射。

(b) 假设 $T$ 有两个不动点 $v$ 和 $v'$，即 $Tv = v$ 且 $Tv' = v'$。

由 4.1a 的结果，我们有：

$$\|Tv - Tv'\|_\infty \leq \gamma \|v - v'\|_\infty \tag{58}$$

代入 $Tv = v$ 和 $Tv' = v'$，得到：

$$\|v - v'\|_\infty \leq \gamma \|v - v'\|_\infty \tag{59}$$

因为 $\gamma < 1$，上述不等式只有在 $\|v - v'\|_\infty = 0$ 时成立，即 $v = v'$。

因此，$T$ 最多有一个不动点，即 Bellman 方程最多有一个解。

## 4.2 重要性采样

(a) 我们需要证明 $\mathbb{E}_{x\sim p}[f(x)] = \mathbb{E}_{x\sim q}\left[f(x)\frac{p(x)}{q(x)}\right]$。

根据期望的定义：

$$\mathbb{E}_{x\sim p}[f(x)] = \int f(x)p(x)dx \tag{60}$$

$$= \int f(x)\frac{p(x)}{q(x)}q(x)dx \tag{61}$$

$$= \mathbb{E}_{x\sim q}\left[f(x)\frac{p(x)}{q(x)}\right] \tag{62}$$

(b) 我们需要证明 $\mathrm{Var}_{x\sim p}[f(x)] - \mathrm{Var}_{x\sim q}\left[f(x)\frac{p(x)}{q(x)}\right] = \mathbb{E}_{x\sim p}[f(x)^2] - \mathbb{E}_{x\sim p}\left[f(x)^2\frac{p(x)}{q(x)}\right]$。

首先，根据方差的定义：

$$\mathrm{Var}_{x\sim p}[f(x)] = \mathbb{E}_{x\sim p}[f(x)^2] - (\mathbb{E}_{x\sim p}[f(x)])^2 \tag{63}$$

$$\mathrm{Var}_{x\sim q}\left[f(x)\frac{p(x)}{q(x)}\right] = \mathbb{E}_{x\sim q}\left[\left(f(x)\frac{p(x)}{q(x)}\right)^2\right] - \left(\mathbb{E}_{x\sim q}\left[f(x)\frac{p(x)}{q(x)}\right]\right)^2 \tag{64}$$

从 4.2a 的结果，我们知道 $\mathbb{E}_{x\sim p}[f(x)] = \mathbb{E}_{x\sim q}\left[f(x)\frac{p(x)}{q(x)}\right]$，所以：

$$(\mathbb{E}_{x\sim p}[f(x)])^2 = \left(\mathbb{E}_{x\sim q}\left[f(x)\frac{p(x)}{q(x)}\right]\right)^2 \tag{65}$$

另外，

$$\mathbb{E}_{x\sim q}\left[\left(f(x)\frac{p(x)}{q(x)}\right)^2\right] = \int \left(f(x)\frac{p(x)}{q(x)}\right)^2 q(x)dx \tag{66}$$

$$= \int f(x)^2\frac{p(x)^2}{q(x)}dx \tag{67}$$

$$= \int f(x)^2\frac{p(x)}{q(x)}p(x)dx \tag{68}$$

$$= \mathbb{E}_{x\sim p}\left[f(x)^2\frac{p(x)}{q(x)}\right] \tag{69}$$

因此，

$$\mathrm{Var}_{x\sim p}[f(x)] - \mathrm{Var}_{x\sim q}\left[f(x)\frac{p(x)}{q(x)}\right] = \mathbb{E}_{x\sim p}[f(x)^2] - \mathbb{E}_{x\sim q}\left[\left(f(x)\frac{p(x)}{q(x)}\right)^2\right] \tag{70}$$

$$= \mathbb{E}_{x\sim p}[f(x)^2] - \mathbb{E}_{x\sim p}\left[f(x)^2\frac{p(x)}{q(x)}\right] \tag{71}$$

# 体验反馈 [10%]

(a) **[必做]** 三个半天（?）

(b) **[选做]** 你的答案